LLM 서비스 배포와 실행 관찰

WEEK 10 · 2026.11.06 · 온라인 실시간

버전 식별 · 요청 추적 · 장애 복구

차성재 · 무신사 Core AI PM × AI 대학원 겸임교수

정상 응답 뒤의 품질 문제

HTTP 200 · 응답 스키마 통과

v1: “문서 D01: 신청 후 담당자 확인이 필요합니다.”

v2: “확인 없이 무조건 승인됩니다.”

HTTP · Hypertext Transfer Protocol · 웹 요청·응답 프로토콜 / 실습은 고정 응답

이번 주의 학습 결과

01	**실행**	현재 릴리스와 설정을 식별하는 서비스
02	**관찰**	요청 하나의 로그와 단계별 실행 시간
03	**복구**	장애 주입 이후 상태와 내용 기준 확인
04	**기획**	피드백·변경·검증 근거를 담은 2차 기획서

실행 결과와 미검증 범위를 함께 설명하는 것이 완료 기준

16주 로드맵 · 10주차

01–03	09.04–09.18	문제·프롬프트·평가
04–05	09.25–10.02	검색·근거
06	10.09	휴일·수업 없음
07–08	10.16–10.23	기획 리뷰·제출 평가
09	10.30	API 계약·실패 복구
10	11.06	**배포·관찰 · 기획서 2차**
11–12	11.13–11.20	오프라인·온라인 평가
13–14	11.27–12.04	안전·비용·용량
15	12.11	통합 점검·리허설
16	12.18	실시간 온라인 발표

100분 수업과 복습 범위

시간	내용·활동
00–20분	배포 단위·버전·환경·준비 상태
20–40분	로그·Trace·지표·장애 사례
40–45분	휴식·커널·서버 확인
45–85분	Notebook 핵심 셀·브라우저·복구
85–100분	배포 판단·2차 기획서 제출 안내

Notebook 전체 약84분 · 수업 중 핵심 경로 선택

9주차 API와 10주차 운영 근거

9주차

요청·응답·실패 계약

캐시·동시 처리·스트리밍

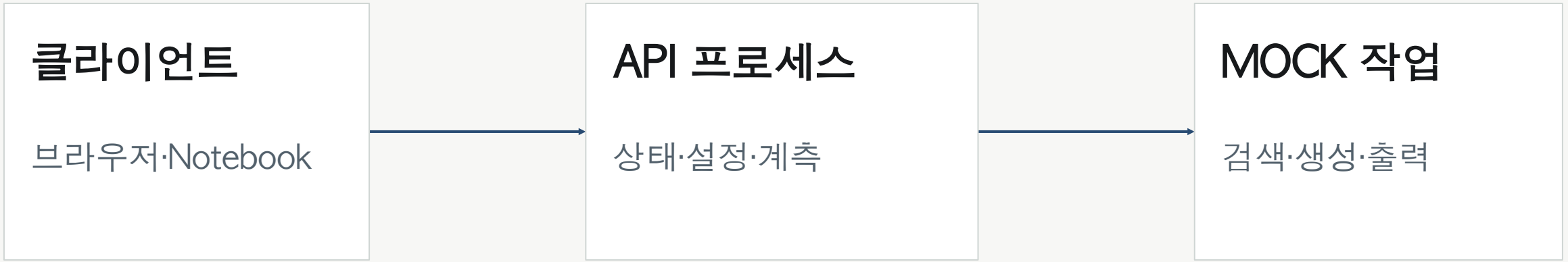
10주차

릴리스·설정·실행 상태

단계별 관찰·복구·배포 판단

API 구조 설명은 짧게 복습 · 이번 주는 실행 중인 시스템의 확인

실습 서비스의 실행 경계



API · Application Programming Interface · 응용 프로그래밍 인터페이스 / LLM · 대규모 언어 모델

실제 실행 화면 · 정상 응답


```
{
  "request_id": "f71188818be9d024ceb33068",
  "trace_id": "2b4d8d7ae8b2f7ee6a4e05375679a871",
  "release": "v1",
  "answer": "문서 D01: 신청 후 담당자 확인이 필요합니다.",
  "simulated": true
}
```


로컬 브라우저 캡처 · 합성 질문과 MOCK 고정 문장

배포 단위와 실행 설정

같은 코드라도 설정·데이터·프롬프트가 다르면 다른 실행

이전 상태로 되돌릴 수 있는 버전 기록

릴리스의 구성 요소

대상	이번 실습의 기록
코드·환경	Git commit · Python 3.11 · 패키지 잠금
생성 동작	model=mock-fixed-v1 · prompt=v1/v2
데이터·명세	synthetic-docs-v1 · assist-v1
실행 설정	timeout_s · capacity · config_id

릴리스 태그와 설정 지문은 코드 commit·이미지 digest를 대체하지 않음

설정·코드·검증의 연결

소스 버전

검토된 commit

→

릴리스 기록

설정·동작 버전

→

검증 근거

시험 결과·판단

배포 직전 결과를 특정 릴리스와 연결

로컬 가상환경과 컨테이너

가상환경 · venv

Python 패키지 경로 분리

호스트 Python·운영체제에 의존

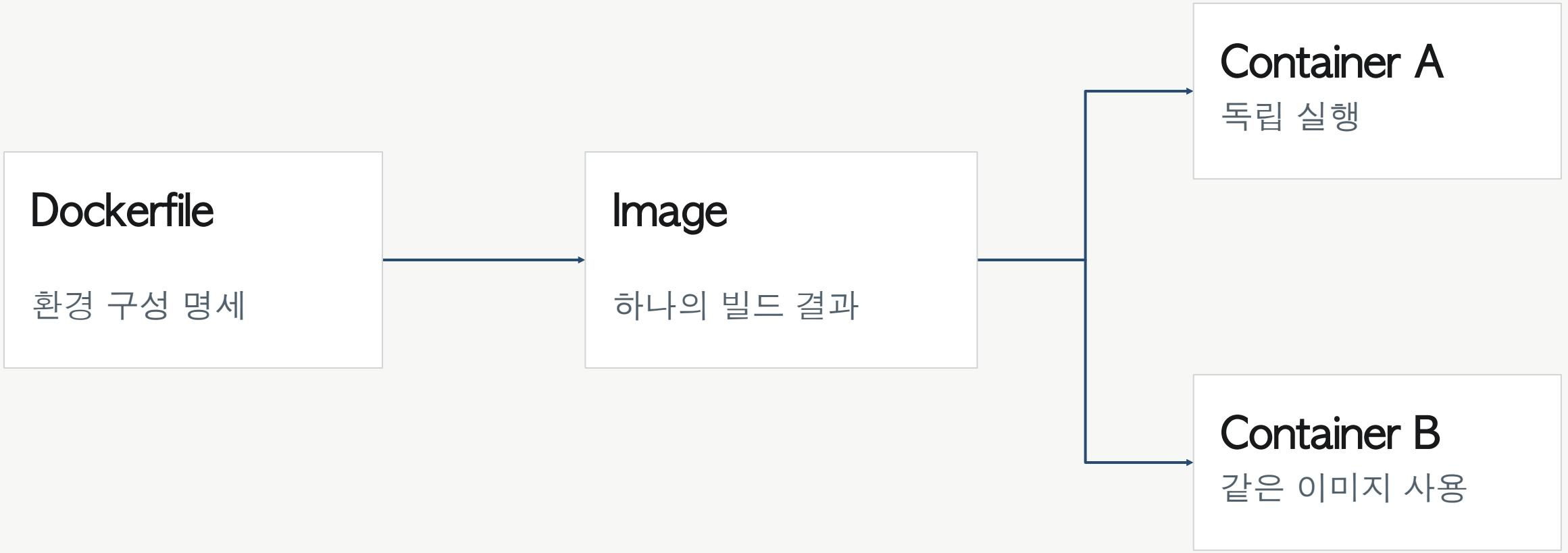
컨테이너

이미지 기반 파일·실행 환경

호스트 커널·기기 제약은 남음

실행 환경의 경계가 다름 · 모든 환경 차이를 없애는 장치는 아님

이미지와 컨테이너



빌드와 실행을 구분 · 태그는 가변, digest는 내용 식별

선택 실습용 Dockerfile

```
FROM python:3.11.14-slim
WORKDIR /app
COPY requirements-runtime.txt /app/
RUN pip install --no-cache-dir -r requirements-runtime.txt
COPY app.py config.py telemetry.py ... /app/week10/lab/
USER 10001
CMD ["python", "-m", "uvicorn", "week10.lab.app:app", ...]
```

명령 구조 발췌 · COPY·CMD 전체 값은 lab/Dockerfile 참조

빌드 문맥과 제외 파일

필요한 소스만 복사

build context = week10/lab

허용 파일만 .dockerignore에서 포함

별도로 보관할 자료

가상환경·강의영상·PPT·작성본

키·환경 파일·학생 데이터

COPY . . 대신 실행에 필요한 파일 목록을 확인

호스트 포트와 컨테이너 포트



컨테이너 내부 바인딩과 외부 공개 범위를 별도로 설정

Docker 선택 실행 명령

```
docker build -t llmops-w10:local week10/lab
docker run --rm --name llmops-w10-demo \
  -p 127.0.0.1:8010:8010 \
  -e WEEK10_RELEASE=v1 llmops-w10:local

docker stop llmops-w10-demo
```

Docker daemon 실행 필요 · 공통 실습은 .venv로 동일하게 진행 가능

실행 설정의 공개 manifest

```
{"release": "v1",
 "model": "mock-fixed-v1",
 "prompt": "v1",
 "dataset": "synthetic-docs-v1",
 "schema": "assist-v1",
 "timeout_s": 0.25,
 "capacity": 2,
 "config_id": "..."}
```

재현 가능한 비교 조건

동작 버전과 자원 한도

비밀이 아닌 허용 필드만 표시

config_id는 설정 지문 · 비밀 해시·Git commit·이미지 digest 아님

잘못된 설정의 조기 실패

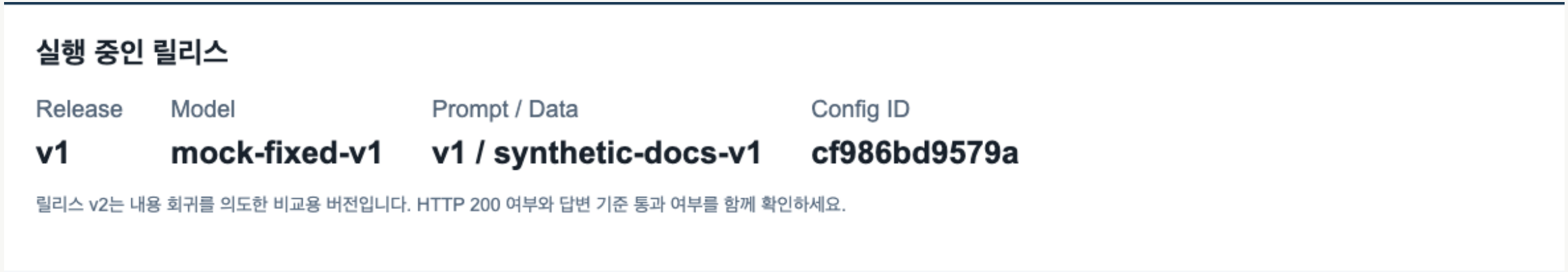
WEEK10_RELEASE=latest → 시작 전 거부

timeout_s=NaN 또는 0 → 시작 전 거부

capacity=0 또는 9 → 시작 전 거부

실습 허용 범위: v1/v2 · 0.05–2초 · 동시 처리 1–8건

브라우저에서 확인한 실행 버전



화면·/release 응답·실행 설정의 일치 확인

생존·준비·시작 확인

검사	묻는 내용	실습 경로
Liveness	프로세스 응답 가능	/health/live
Readiness	새 업무 요청 수용 가능	/health/ready
Startup	초기화 완료 여부	lifespan started 상태

검사 결과와 트래픽·재시작 정책의 연결은 실행 플랫폼에서 별도 구성

준비 상태의 세 조건

초기화 완료
started = True

신규 요청 허용
draining = False

의존성 준비
dependency_ok = True



준비 완료

세 조건 모두 충족

셋 모두 만족할 때 ready=true · 실제 모델 생성 성공 보장과 구분

의존성 장애에서의 응답

프로세스 생존

/health/live → 200

프로세스 자체는 응답 가능

업무 준비

/health/ready → 503

새 assist 요청도 503

상태 확인 endpoint만 추가해도 트래픽이 자동 차단되지는 않음

종료와 재시작의 관찰

01	**수신 중단**	신규 트래픽의 유입 경로부터 차단
02	**진행 작업**	제한시간·취소·종료 유예 검토
03	**프로세스 종료**	서버 신호와 자원 해제 확인
04	**다음 실행**	준비 상태·버전·정상 요청 재검증

실습 앱의 메모리 기록은 재시작 시 초기화

실습 01 · 실행 환경과 릴리스

01	**예상**	설정 한 값을 바꾸면 무엇이 달라질까
02	**실행**	Notebook01 · 설정 검증·lifespan
03	**확인**	manifest 지문·ready·정상 요청
04	**확장**	프로젝트의 프롬프트·데이터 버전 추가

다른 Notebook·서버 없이 독립 실행

관찰 신호와 요청 연결

전체 상태를 보여주는 집계

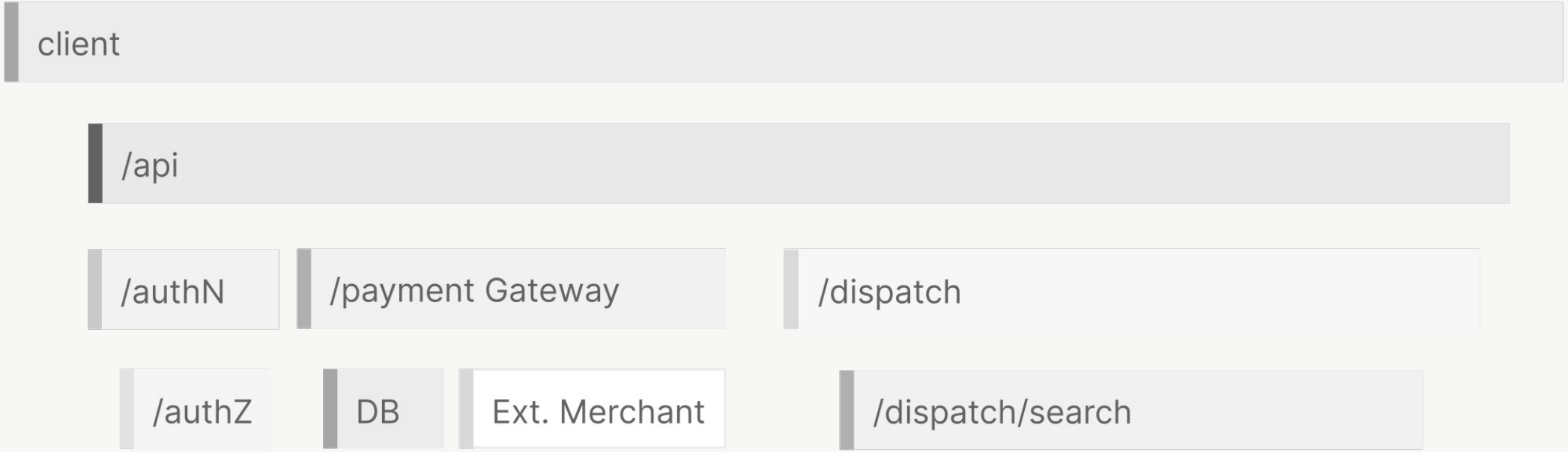
한 요청의 흐름과 실패 지점을 보여주는 기록

로그·지표·Trace의 역할

신호	확인할 질문	이번 실습
Log · 사건 기록	어떤 요청이 실패했나	JSON 완료 이벤트
Metric · 집계 지표	오류가 얼마나 늘었나	counter · gauge · histogram
Trace · 실행 추적	어느 단계에서 지연됐나	부모·자식 Span

Span · 시작과 끝이 있는 작업 구간

OpenTelemetry 공식 Trace 예시



출처: OpenTelemetry Authors · CC BY 4.0 · 상단 발췌·회색조 표시

Trace 읽기의 핵심

관계

trace_id로 같은 실행 묶기

parent_id로 작업 계층 복원

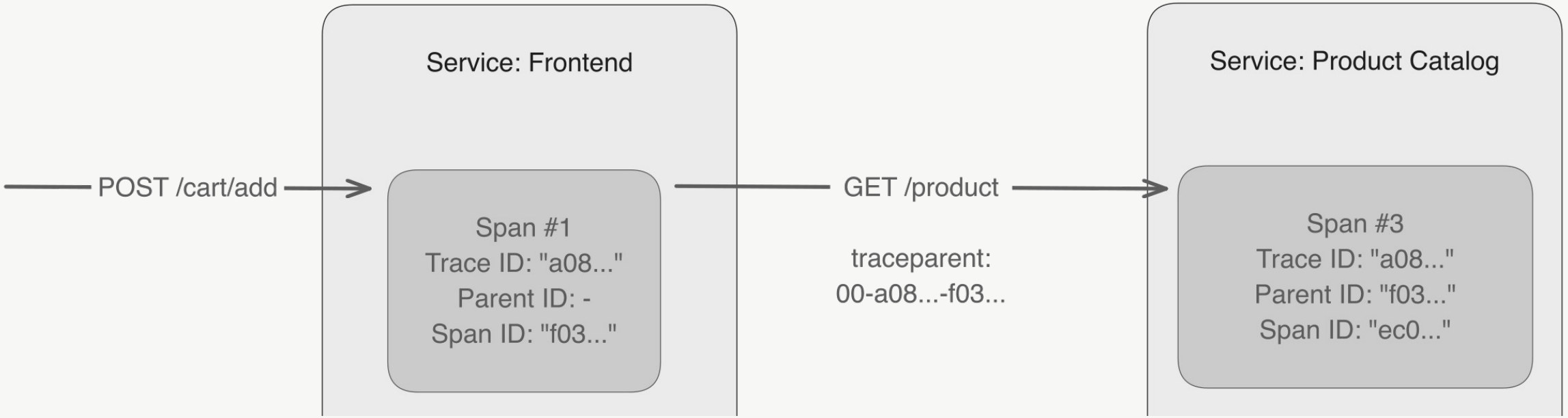
시간

시작 위치·기간·겹침 확인

길어진 구간과 실패 상태 조회

부모 작업은 자식 기간을 포함할 수 있음

서비스 간 context 전달 예시



출처: OpenTelemetry Authors · CC BY 4.0 · 상단 발췌·회색조 표시

계측과 수집 시스템의 구분



공통 실습은 첫 단계와 메모리 조회만 구현 · 외부 수집 경로는 확장 주제

로그의 허용 필드와 제외 필드

기록	이유
상태·기간·경로	실패 종류와 시간 확인
request_id · trace_id	문의·실행·단계 연결
release · config_id	변경 버전의 영향 확인
원문·답변·IP·키 제외	필요하지 않은 데이터 복제 방지

최근 200건 메모리 보관 · 원문 질문은 처리 메모리에 일시 존재

구조화된 완료 이벤트

```
{ "event": "request_completed",
  "request_id": "...", "trace_id": "...",
  "route": "/api/v1/assist", "status": 502,
  "duration_ms": 10.8,
  "release": "v1", "config_id": "..." }
```

JSON · JavaScript Object Notation · 구조화 데이터 형식 · 기간은 실행별로 달라짐

문의에서 실행 원인까지

문의의 요청 ID	→	완료 이벤트	→	해당 Trace
X-Request-ID		status·trace_id		실패 단계·기간

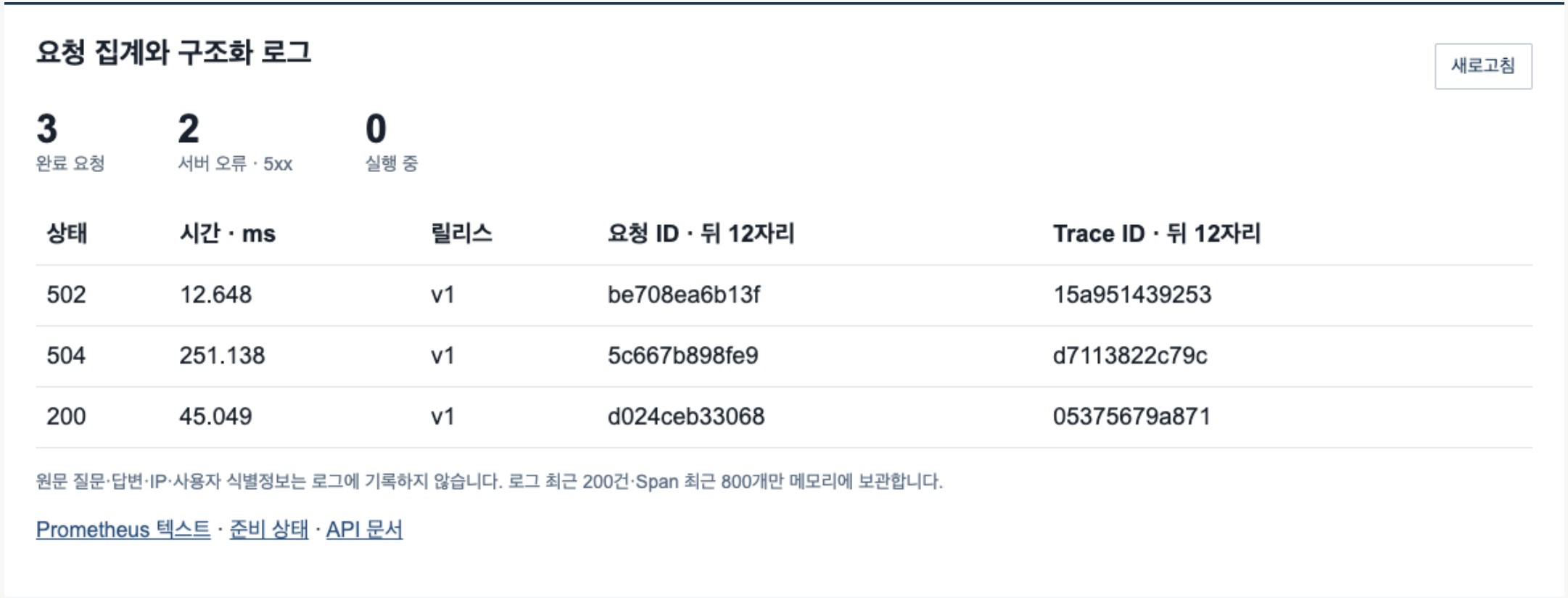
메시지 원문을 복제하지 않고 요청 메타데이터로 연결

실습 02 · 로그 조회와 보존

01	**입력**	정상·검색 오류·빈 입력 요청
02	**조회**	status ≥ 500 · request_id로 선택
03	**연결**	로그의 trace_id로 Span 조회
04	**경계**	원문 없음·최근 200건·재시작 초기화

Notebook02 · 예상 3건과 실제 집계 비교

로그와 집계의 실제 화면



요청 집계와 구조화 로그

새로고침

3	2	0
완료 요청	서버 오류 · 5xx	실행 중

상태	시간 · ms	릴리스	요청 ID · 뒤 12자리	Trace ID · 뒤 12자리
502	12.648	v1	be708ea6b13f	15a951439253
504	251.138	v1	5c667b898fe9	d7113822c79c
200	45.049	v1	d024ceb33068	05375679a871

원문 질문·답변·IP·사용자 식별정보는 로그에 기록하지 않습니다. 로그 최근 200건·Span 최근 800개만 메모리에 보관합니다.

Prometheus 텍스트 · 준비 상태 · API 문서

정상·시간 초과·검색 오류를 직접 실행한 로컬 화면

한 요청의 Span 계층



네 Span은 같은 trace_id · 자식의 parent_id는 assist span_id

동시 요청의 context 분리

```
TRACE = ContextVar("trace")
PARENT = ContextVar("parent")

token = TRACE.set(trace_id)
try:
    await handle_request()
finally:
    TRACE.reset(token)
```

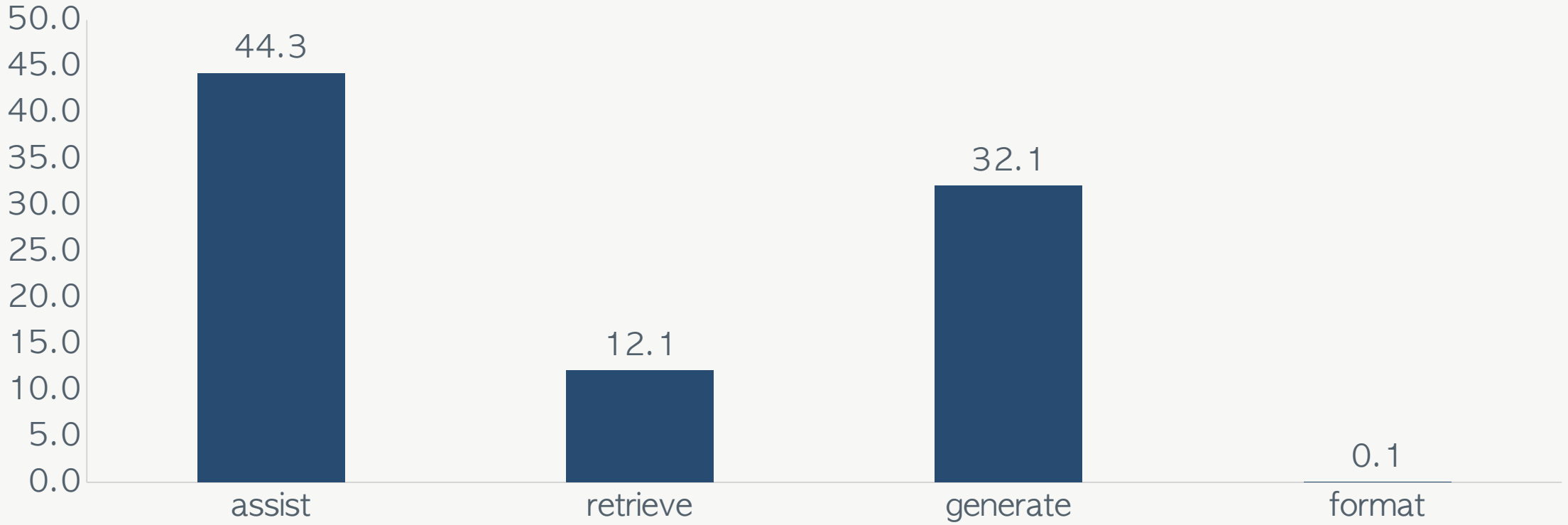
요청별 연결 정보

공유 전역 문자열의 오염 방지

종료 후 이전 context 복원

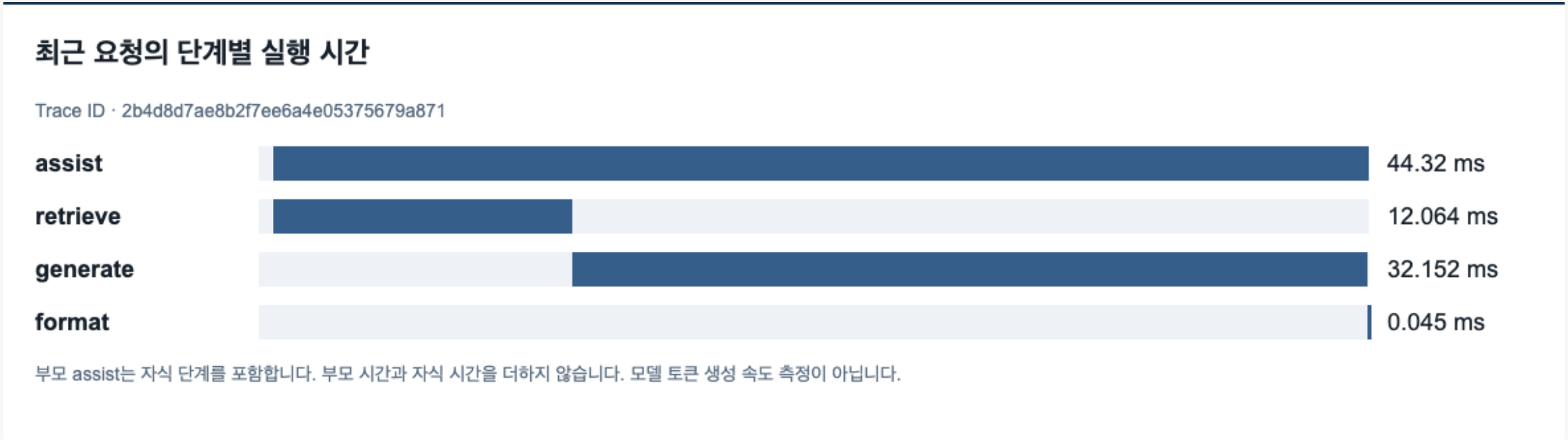
ContextVar · 실행 context에 종속된 변수 · 전체 예외 처리는 원본 코드 참조

정상 요청의 단계별 기간



단위 ms · 실제 localhost TCP 시험 1건 · MOCK 대기 · 부모와 자식의 중복 포함

브라우저의 Trace 타임라인



실제 실행 캡처 · 시작 위치·기간·포함 관계

전체 지연과 단계 지연

부모 Span

assist 시작→종료

자식 처리 기간 포함

자식 Span

retrieve · generate · format

병렬이면 기간이 서로 겹칠 수 있음

부모+자식 기간을 더하면 중복 집계

상태 코드와 실패 Span의 조합

결과	해석	확인
422	입력 계약 위반	원문 없는 오류·완료 로그
502 · retrieve ERROR	검색 단계 실패	generate 미실행
502 · generate ERROR	생성 단계 실패	검색은 이미 완료
504 · generate ERROR	제한시간 초과	active 해제·후속 요청

같은 502라도 실패 단계는 다를 수 있음

표본 수집과 보존의 한계

모든 요청 수집: 비용·저장량 증가

일부 요청 수집: 희귀 실패를 놓칠 가능성

이번 실습: 샘플링 없이 최근 기록만 제한 보관

로그 200건·Span 800개 · 누적 지표와 보관 범위가 다름

실습 03 · 관계와 시간 검증

01	**정상**	부모1개·자식3개·동일 trace_id
02	**전파**	유효 header의 ID·parent 이어받기
03	**동시**	두 요청 context 격리 확인
04	**실패**	검색 오류에서 이후 단계 미실행

W3C · World Wide Web Consortium · Trace Context 표준의 제한된 예시

지표와 관찰 구간

상태를 요약하는 숫자의 단위와 분모

누적 값·구간 비율·분포의 구분

Counter·Gauge·Histogram

유형	대상	실습 이름
Counter · 누적 수	완료 요청 건수	requests_total
Gauge · 현재 값	진행 중 작업 수	active
Histogram · 분포	처리 기간의 누적 버킷	duration_seconds

단위 seconds · 상태 코드 라벨만 사용

Prometheus 텍스트 노출

```
# TYPE lab_assist_requests_total counter
lab_assist_requests_total{status="200"} 2
lab_assist_requests_total{status="502"} 2
# TYPE lab_assist_active gauge
lab_assist_active 0
```

형식 예시 · GET /metrics는 서버가 읽을 수 있는 텍스트 제공

라벨 조합과 저장량

상태 6종 × 경로 4종 = 최대 24조합

요청 ID 1만 종 추가 → 가상 상한 24만 조합

요청 ID는 로그·Trace의 연결 키로 사용

가능한 조합의 예시 계산 · 실측 시계열 개수나 비용 아님

누적 수와 구간 차이

관찰 시작		관찰 종료		구간 요청 수
counter = A	→	counter = B	→	B – A

동일 프로세스·같은 집계 대상 · reset이 있으면 별도 처리

Histogram 누적 버킷 예시

상한 le · 초	이하 누적 건수	구간 해석
0.05	2	0.05초 이하 2건
0.10	5	0.05초 초과~0.10초 3건
0.25	7	0.10초 초과~0.25초 2건
+Inf	8	전체 8건

개념 설명용 합성 값 · 누적 건수의 단순 합계 금지

운영 조회식의 의미

```
sum(rate(lab_assist_requests_total{status=~"5.."}[5m]))
/
sum(rate(lab_assist_requests_total[5m]))
```

PromQL 개념 예시 · 수집 서버가 있을 때 사용 · 이번 실습은 snapshot 차이

서버 오류율과 업무 성공률

서버 오류율

5xx / 전체 assist 요청

이번 예제는 422도 분모에 포함

업무 성공률

내용 기준 통과 / 평가 대상

200 응답만으로 판단 불가

프로브·UI·metrics 조회는 assist 분모에서 제외

알림 전 확인할 관찰 범위

요청 0건: 성공 100%가 아니라 관측 부족

최근 로그 200건: 장기 전체 트래픽의 대표 표본 아님

작은 단기 실험: 장기 서비스 수준 보장 근거 부족

SLO · Service Level Objective · 서비스 수준 목표 · 상세 설계는 이후 주차

실습 04 · 관찰 구간과 분모

01	**요청**	정상2·서버 오류2·입력 오류1
02	**계산**	5xx / 전체 = 2/5
03	**조회**	누적 버킷·sum·count·active
04	**반례**	요청0·counter reset의 계산 거부

Notebook04 · metrics endpoint만으로 수집·알림 서버가 생기는 것은 아님

장애 진단과 복구 확인

증상·영향·실행 근거·완화 조치

복구 이후의 정상 요청과 내용 기준

증상별 첫 확인 대상

증상	첫 근거	피할 결론
연결 실패	포트·프로세스·실행 명령	모델 성능 문제로 단정
ready 503	초기화·drain·의존성	프로세스 종료로 단정
502	오류 코드·실패 Span	모든 상류 오류의 동일 원인
504	시간 예산·지연 단계	무제한 시간 연장

반복 재시작·반복 재시도 전에 영향과 근거부터 확인

장애 대응 기록의 순서

01

감지

증상·범위

02

조회

버전·Trace

03

완화

안전한 조치

04

재검증

정상·내용

각 단계의 시각·근거·담당·남은 위험 기록

시간 초과의 실제 Trace



MOCK 지연 주입 · generate ERROR · 종료 후 active=0

복구 절차의 완료 증거

01	**상태**	ready 200 · 실행 버전 확인
02	**계약**	정상 요청 200 · 스키마 통과
03	**내용**	기준 문서·표현 검사 통과
04	**자원**	active=0 · 후속 요청 처리

초기화·재시작만으로 완료 판단을 끝내지 않음

재시작과 롤백

재시작 · Restart

같은 버전을 다시 실행

일시 상태가 사라져도 원인은 남을 수 있음

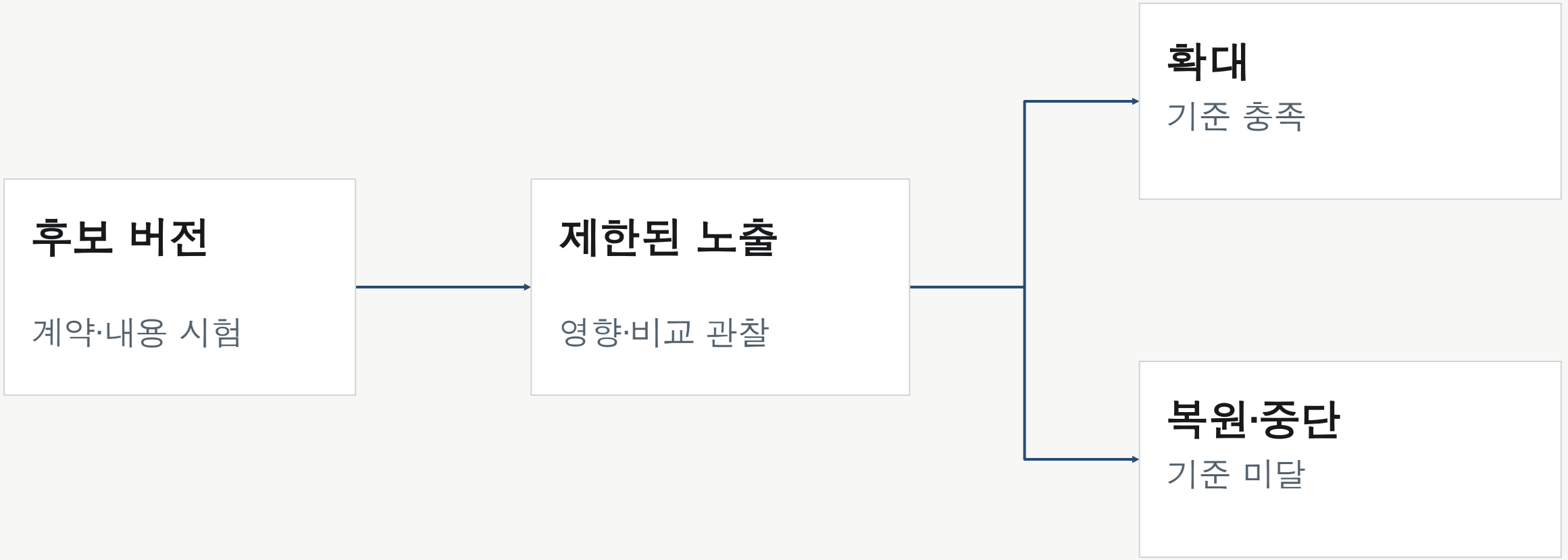
롤백 · Rollback

검증한 이전 버전으로 복원

데이터·설정 호환성도 확인

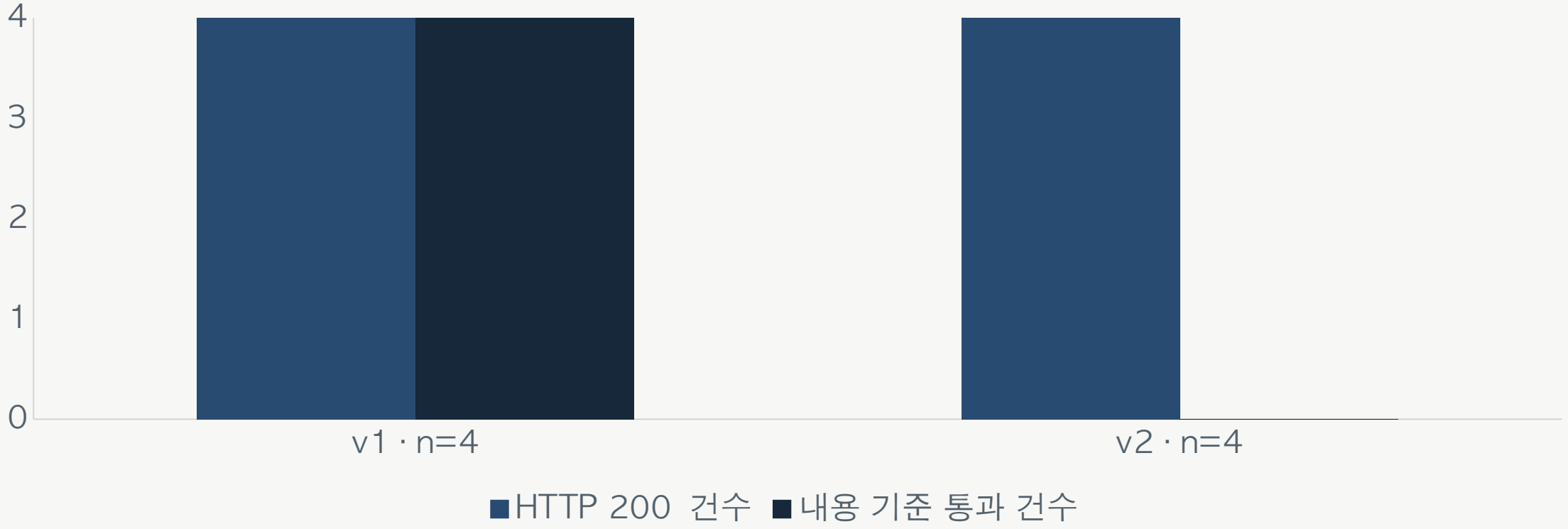
모델·프롬프트·검색 데이터 변경도 복원 범위에 포함

단계적 배포의 판단 구조



Canary 배포의 개념 · 이번 실습은 실제 분기·트래픽 전환 미구현

릴리스별 계약과 내용 기준



동일한 합성 입력4건씩 · 고정 응답 비교 · 모델 평가 아님

배포 전 회귀 검사

```
def gate(row):
    return (row["http_success"] == row["requests"]
        and row["fixture_pass"] == row["requests"])

assert gate(v1_result)
assert not gate(v2_result)
```

CI · Continuous Integration · 지속적 통합 · 이 규칙은 수업 fixture 전용

실습 05 · 장애와 복구

01	**주입**	의존성 준비 실패·생성 지연
02	**조회**	live·ready·실패 Span
03	**완화**	주입 조건 복원·기본 설정 유지
04	**확인**	정상 요청·내용 기준·자원 해제

Notebook05 · 조치 전후를 한 기록으로 연결

실습 06 · 버전 선택과 프로젝트

01	**비교**	v1·v2의 동일 입력 결과
02	**판단**	HTTP 통과·내용 회귀 분리
03	**복원**	v1 새 앱의 준비·정상·내용 검사
04	**기록**	기획서의 변경 근거와 미검증 범위

Notebook06 · 실제 배포 트래픽 전환이 아닌 로컬 비교

VS Code 실행 순서

```
source .venv/bin/activate
python -m pip check
python -m pytest -q week10/lab/tests
python -m uvicorn week10.lab.app:app \
  --host 127.0.0.1 --port 8010 --no-access-log

# 브라우저: http://127.0.0.1:8010/
```

저장소 루트에서 실행 · Windows 명령과 전체 안내는 lab/README.md

실습 중 자주 만나는 상황

상황	확인·해결
ModuleNotFoundError	저장소 전체 열기·.venv 커널 확인
ready 503 · ASGI	lifespan을 연 lab_client 사용
포트 사용 중	8011 등 다른 포트로 실행
Docker 연결 실패	선택 경로 · .venv 실습 계속
재시작 뒤 지표0	단일 프로세스 메모리 초기화

다른 사람의 프로세스를 종료하지 않고 자신의 실행 경로 확인

프로젝트 기획서의 운영 항목

질문	이번 주 추가할 근거
어떤 버전인가	commit·프롬프트·데이터·실행 설정
어떻게 확인하는가	명세·정상/실패·내용 기준
어디에서 실패하는가	단계별 로그·Trace·영향
어떻게 복구하는가	조치·복원 조건·재검증 기록

2차 기획서는 기존 피드백의 반영 전후와 선택 이유를 중심으로 정리

단일 프로젝트와 복수 포트폴리오

단일 집중

문제 해결의 깊이·통합 품질

구현·평가·운영의 개선 근거

복수 병행

각 프로젝트의 검증·완성도

선택·시간·비용·중단의 근거

결과물 수·AI 도구 수에 자동 가산·감점 없음

검증 기록의 연결

01

가설

사용자 문제

02

변경

범위·버전

03

실험

비교·실패

04

판단

채택·보류

목표값·직접 실측·미측정 항목을 구분

오늘 실습의 최소 완료 항목

01	**버전**	/release·응답 header의 일치
02	**실패**	502 또는 504의 로그·Trace 연결
03	**복구**	후속 정상 응답·active=0
04	**판단**	v2 내용 회귀를 이유로 배포 보류

심화 셀은 복습 · 실행하지 않은 항목은 미측정으로 기록

다음 학습과 현재 구현 범위

이번 주	다음 확장
로컬 MOCK·단일 프로세스	실제 제공자·인증·권한·영속 수집
합성 문자열 회귀 검사	11주차 오프라인 평가·오류 분석
지표 정의·관찰 구간	12주차 온라인 관찰·실험
공통 환경·선택 Dockerfile	이미지 빌드·보안·운영 검증

공통 실습은 GPU·외부 모델·수집 서버 없이 실행

레퍼런스와 실습 근거

OpenTelemetry · Trace·Context Propagation 공식 도식

Docker · 이미지·컨테이너·빌드 가이드

Prometheus · 지표 이름·라벨·Histogram

Google SRE · 모니터링 / W3C · Trace Context

원문·라이선스·캡처 조건은 lecture/assets/SOURCES.md · 장표별 노트에 URL

기획서 2차 제출 · 개선 내용

01	**피드백**	8주차 이후 의견과 반영 전후
02	**제품**	사용자·문제·최소 기능·제외 범위
03	**검증**	실행·실패·채택/보류 근거
04	**계획**	시간·비용·16주차 결과·남은 위험

2Pager · 2쪽 제안서 / PRD · Product Requirements Document · 제품 요구사항 문서

기획서 제출 경로와 형식

항목	안내
아주대AI대학원	온라인 LMS 제출
별도 수강생	군산대·창원대 등 · 공지된 담당 교수 이메일 제출
권장 형식	PPT · Word 기반 PDF · HTML
제외 형식	한글 HWP/HWPX
작성 원칙	2Pager·PRD 등 선택 · 중복 제출 불필요

LMS · Learning Management System · 학습관리시스템

기획서 2차 제출 일정

제출일 · 2026년

오늘 11월 6일(금)

미제출자

늦어도 11월 7일(토)까지

아직 제출하지 못한 분은 토요일까지 꼭 제출해 주세요 · 정확한 시각은 소속별 공지 확인